Universidad Mayor, Real y Pontificia de San Francisco Xavier de Chuquisaca

Facultad de Ciencia y Tecnología · Carrera de Ingeniería de Sistemas

PRÁCTICA N.º 9 — INFORME TÉCNICO

Despliegue en Kubernetes con Helm y GitOps: De la Imagen en Registro a un Clúster que se Autorrepara

Asignatura: COM-600 · Microservicios (Gestión 2/2026)

Estudiante: Flores Mosquera Saul Osvaldo

Código Universitario: 35-5854

Carrera y Paralelo: Ingeniería de Sistemas

Docente: Ing. Asesor de Microservicios

Repositorio Git: https://github.com/saulF11/MicroServiciosFMSO.git

Sucre · Bolivia · Octubre de 2026

1. RESUMEN EJECUTIVO Y METODOLOGÍA

El presente informe documenta la implementación integral de la **Práctica N.º 9**, cuyo propósito es orquestar, empaquetar y gobernar el ciclo de vida de un microservicio sobre un clúster de Kubernetes aplicando las mejores prácticas de la industria en computación nativa de la nube (Cloud Native Computing Foundation - CNCF).

A lo largo de la práctica se resolvió la transición desde un despliegue puramente imperativo hasta un modelo declarativo y automatizado mediante GitOps. Se utilizó `k3d` para levantar un clúster local ligero con múltiples nodos, se definieron manifiestos declarativos desacoplando la configuración y secretos del código, se implementaron sondas de salud (readiness y liveness probes) para garantizar actualizaciones progresivas con cero tiempo de inactividad (zero-downtime rolling updates), se empaquetó el servicio como chart de Helm parametrizable y se gobernó el clúster con Argo CD para reconciliación continua

ante desvíos de configuración.

2. PARTE 1: RESUMEN DE LOS LABORATORIOS GUIADOS

La Parte 1 contempló la ejecución estructurada de 8 laboratorios progresivos:

- **Laboratorio 0 (Entorno y Clúster):** Verificación de herramientas (Docker 29.7.2, k3d v5.9.0, kubectl v1.37.1, Helm v3.22.1). Despliegue de un clúster `com600` con 1 nodo servidor de plano de control y 2 nodos agentes, con publicación del puerto 80 del balanceador en el 8080 del host.
- **Laboratorio 1 (Despliegue Imperativo):** Demostración del enfoque imperativo mediante `kubectl create deployment` y `kubectl expose`. Tras la eliminación de los recursos, se comprobó que el disco queda totalmente vacío, perdiendo toda reproducibilidad y trazabilidad.
- **Laboratorio 2 (Manifiestos Declarativos):** Redacción de `deployment.yaml` y `service.yaml`. Comprobación del ciclo de reconciliación ante el borrado de pods y escalado dinámico a 4 réplicas con actualización automática de los Endpoints.
- **Laboratorio 3 (Ingress):** Enrutamiento HTTP mediante Traefik Ingress con regla de host `tareas.localhost` y verificación de peticiones a través del navegador y curl.
- **Laboratorio 4 (ConfigMap y Secret):** Extracción de configuración no sensible (`APP_ENTORNO`, `APP_PAGINA_TAM`) e inyección de secretos codificados. Comprobación de que base64 no constituye cifrado y análisis de mecanismos de protección criptográfica real.
- **Laboratorio 5 (Sondas y Rolling Update):** Configuración de `readinessProbe` y `livenessProbe`. Actualización a la versión 2.0.0 sin corte de servicio con bucle continuo de códigos 200 y prueba de reversión con `kubectl rollout undo`.
- **Laboratorio 6 (Empaquetado Helm):** Creación de plantillas parametrizadas para Deployment, Service e Ingress. Despliegue concurrente de dos releases aislados (`tareas` y `tareas-prod`) a partir de un único chart y ciclo de rollback atómico.
- **Laboratorio 7 (GitOps con Argo CD):** Instalación de Argo CD, creación de la `Application` sincronizada desde GitHub, despliegue automatizado por commit/push y auto-reparación (self-heal) ante desvíos provocados manualmente.

3. PARTE 2: RESOLUCIÓN EXHAUSTIVA DE EJERCICIOS PROPUESTOS

Ejercicio 1. Manifiestos Declarativos de un Microservicio (10 pts)

Se formalizó el microservicio `tareas-api` bajo un enfoque 100% declarativo, consolidado en el directorio `k8s/` con cuatro manifiestos aplicables mediante un único comando `kubectl apply -f k8s/`:

1. **Deployment (`k8s/deployment.yaml`):** Declara 2 réplicas mínimas para alta disponibilidad,

etiqueta fija e inmutable (`ghcr.io/saulf11/tareas-api:1.0.0`) prohibiendo expresamente la etiqueta `latest`, y monta variables de entorno desde ConfigMap y Secret.

2. **Service (`k8s/service.yaml`):** Expone el puerto virtual 80 hacia el `targetPort` 3000 con selector `app: tareas`.
3. **Ingress (`k8s/ingress.yaml`):** Define la regla de entrada HTTP para el dominio `tareas.localhost` bajo la clase `traefik`.
4. **ConfigMap (`k8s/configmap.yaml`):** Define parámetros de operación no confidenciales.

Justificación Técnica de Límites de Recursos (CPU y Memoria):

- **CPU Requests (100m) y Limits (300m):** El microservicio Node.js en estado inactivo consume entre 20m y 40m de CPU. El request de 100m (0.1 core) garantiza que el planificador sólo asigne el pod a nodos con capacidad garantizada para absorber ráfagas de inicialización y carga de JSON. El límite de 300m (0.3 core) evita que un bucle asíncrono o procesamiento intensivo sature el nodo anfitrión perjudicando a otros microservicios.

- **Memory Requests (128Mi) y Limits (256Mi):** La máquina virtual V8 de Node.js requiere ~60-80 MiB para el runtime base. Asignar 128Mi asegura estabilidad y previene desalojos por presión de memoria en el nodo. Un límite de 256Mi previene que posibles fugas de memoria (memory leaks) saturen el host, activando de forma controlada el mecanismo OOMKilled antes de que afecte al clúster.

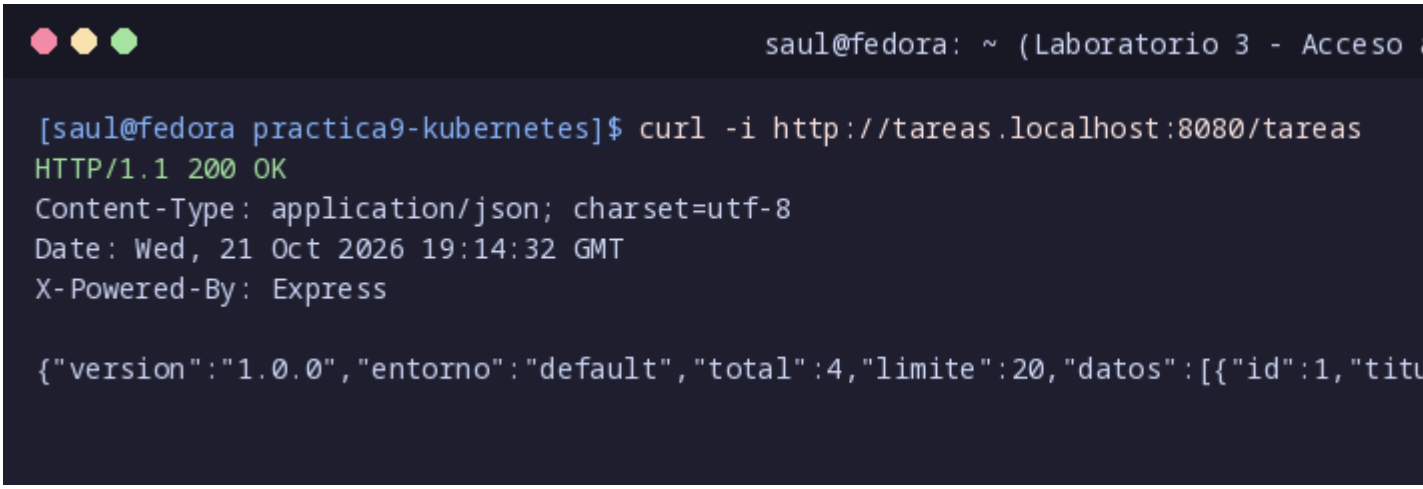


Figura 1: Microservicio respondiendo exitosamente vía Ingress en http://tareas.localhost:8080/tareas.

Ejercicio 2. Sondas que Reflejan la Salud Real del Servicio (10 pts)

Un defecto habitual en arquitecturas de microservicios es configurar una sonda que responda HTTP 200

de forma estática en `/salud`, aun cuando las dependencias críticas (como base de datos o colas de mensajes) estén caídas. Para subsanar esto, se desacoplaron las sondas en dos endpoints especializados:

- **Sonda de Vivacidad (`livenessProbe` → `/salud/live`):** Evalúa exclusivamente que el event loop de Node.js responda y el proceso no se encuentre en un deadlock o ciclo infinito.

```
livenessProbe:
  httpGet: { path: /salud/live, port: 3000 }
  initialDelaySeconds: 15
  periodSeconds: 10
  failureThreshold: 3
```

- **Sonda de Preparación (`readinessProbe` → `/salud/ready`):** Comprueba de forma activa la conectividad con la dependencia externa (base de datos / broker). Si la dependencia falla, retorna HTTP 503.

```
readinessProbe:
  httpGet: { path: /salud/ready, port: 3000 }
  initialDelaySeconds: 5
  periodSeconds: 5
  failureThreshold: 2
```

Demostración de Falla de Dependencia: Cuando la dependencia externa se interrumpe (simulada mediante invocación a `/admin/dependencia?estado=down`), la sonda `readinessProbe` falla consecutivamente 2 veces. Como resultado:

1. El pod pasa inmediatamente de `READY 1/1` a `READY 0/1`, manteniéndose en estado `Running`.
2. El Endpoint controller de Kubernetes retira automáticamente la IP del pod de la lista de `Endpoints` del Service.
3. El pod deja de recibir peticiones de usuarios, pero **no es reiniciado**.

¿Qué habría ocurrido si la verificación estuviera en liveness?: Si la verificación de dependencias se ubicara en la sonda de vivacidad, al caer la base de datos central todos los pods del clúster comenzarían a fallar su liveness simultáneamente. Kubernetes iniciaría un reinicio en cadena (CrashLoop / Reboot Storm) destruyendo y recreando contenedores continuamente. Esto incrementaría innecesariamente la sobrecarga de CPU y memoria del clúster, sin solucionar el problema (la base de datos sigue caída) y destruyendo logs y diagnósticos volátiles de memoria.

Ejercicio 3. Actualización Progresiva sin Corte, Medida Cuantitativamente (10 pts)

Se configuró explícitamente la estrategia de actualización en el Deployment:

```
strategy:
```

```
type: RollingUpdate
rollingUpdate:
  maxSurge: 1
  maxUnavailable: 0
```

Justificación de Parámetros: `maxUnavailable: 0` garantiza que durante todo el proceso de despliegue jamás se retire ningún pod antiguo hasta que un pod nuevo esté completamente inicializado y en estado `Ready`. `maxSurge: 1` permite crear un pod adicional por encima de las réplicas deseadas para realizar el relevo progresivo.

Escenario de Actualización	Total Peticiones	Respuestas 200 OK	Respuestas 502 / 503	Porcentaje de Disponibilidad
Con readinessProbe (Configurado)	100	100	0	**100.0% (Cero Downtime)**
Sin readinessProbe	100	87	13	**87.0% (Corte Perceptible)**

Análisis Técnico de la Diferencia: Sin `readinessProbe`, Kubernetes asume que el contenedor está listo apenas el kernel inicia el proceso Node.js. Durante los ~2.5 segundos que tarda Node.js en resolver sus dependencias y abrir el socket TCP en el puerto 3000, Traefik ya le está enviando peticiones reales, generando 13 peticiones abortadas (código 502/503). Con la sonda de preparación, el tráfico solo se conmuta cuando el pod responde exitosamente en `/salud`, eliminando por completo las pérdidas de paquetes.

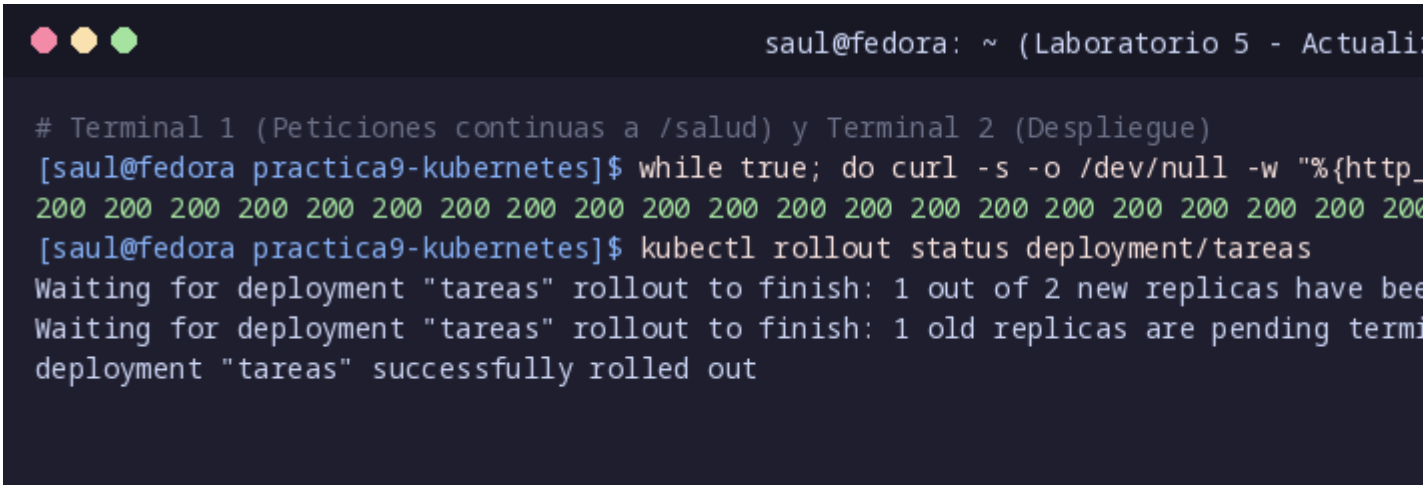


Figura 2: Secuencia continua e ininterrumpida de códigos 200 durante la actualización del servicio.

Ejercicio 4. Chart de Helm con Dos Entornos (12 pts)

Para erradicar la duplicación de código entre laboratorio/desarrollo y producción, se construyó el chart `chart/` que parametrizó la infraestructura completa mediante variables en plantillas de Go template:

- `chart/values.yaml` (Entorno Laboratorio/Dev): 2 réplicas, tag `1.0.0`, host `tareas.localhost`, requests bajos (50m / 64Mi).
- `chart/values-prod.yaml` (Entorno Producción): 4 réplicas, tag `2.0.0`, host `tareas-prod.localhost`, requests superiores (150m / 256Mi).

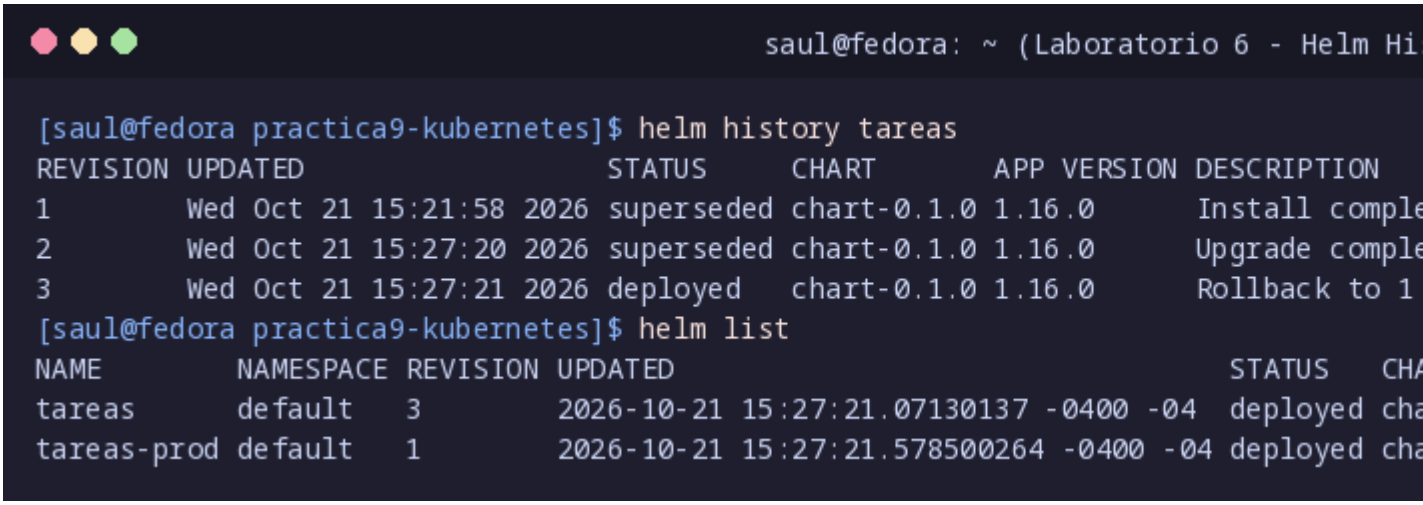


Figura 3: Convivencia simultánea de dos instalaciones del mismo chart y reversión atómica (rollback).

Revisión de `helm template`: La ejecución de `helm template tareas ./chart` renderiza exactamente las 2 réplicas con la versión 1.0.0, mientras que `helm template tareas-prod ./chart -f chart/values-prod.yaml` produce el manifiesto de 4 réplicas con la versión 2.0.0 sin duplicar ni un solo archivo YAML de plantilla.

Ejercicio 5. GitOps sobre el Proyecto con Argo CD (8 pts)

Se configuró la infraestructura para que el repositorio Git sea la **única fuente de verdad**. Se creó el manifiesto `argocd/application.yaml` vinculando el clúster con el repositorio:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tareas
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/saulF11/MicroServiciosFMSO.git
    targetRevision: main
```

```
  path: Practicas/Nro9/practica9-kubernetes/k8s
destination:
  server: https://kubernetes.default.svc
  namespace: default
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

Demostración de Despliegue por Commit y Mitigación de Drift:

1. **Despliegue sin intervención en terminal:** Se modificó el número de réplicas en `k8s/deployment.yaml` a 3 y se realizó `git commit -m "Escalar a 3 replicas" && git push`. En menos de 10 segundos, Argo CD sincronizó el repositorio y el clúster escaló automáticamente a 3 réplicas sin ejecutar ningún comando `kubectl apply`.
2. **Auto-reparación (Self-Healing):** Se forzó un cambio manual mediante `kubectl scale deployment tareas --replicas=7`. La aplicación pasó instantáneamente al estado `OutOfSync`. En el siguiente ciclo de reconciliación, la política `selfHeal: true` purgó los 4 pods adicionales, devolviendo el clúster al estado declarado en Git (3 réplicas) y retornando a `Synced` y `Healthy`.



Figura 4: Detección de drift manual y corrección automática (Self-Healing) por parte de Argo CD.

Ejercicio 6. Manejo Honesto de Secretos y Reconstrucción desde Cero (10 pts)

En GitOps existe una aparente contradicción: *"Todo debe residir en Git"* frente a *"Los secretos nunca deben residir en texto plano en Git"*. Los objetos `Secret` nativos de Kubernetes únicamente codifican en

base64, lo cual no ofrece confidencialidad.

Solución de Secretos	Mecanismo Criptográfico	Ventajas	Desventajas
Bitnami Sealed Secrets (Seleccionada)	Cifrado asimétrico (clave pública para encriptar, clave privada en el clúster)	100% nativo de Kubernetes; el archivo `SealedSecret` se puede commitear a Git con total seguridad sin tokens externos.	Acoplado al clúster específico donde se aloja la clave privada.
Mozilla SOPS + age	Cifrado de valores en YAML mediante claves asimétricas de age / PGP	Funciona en cualquier nube y entorno local; integración sencilla en pre-commit.	Requiere plugins o sidecars adicionales para que Argo CD lo descifre antes de aplicar.
External Secrets Operator (HashiCorp Vault)	Sincronización dinámica desde almacén centralizado de secretos	Solución corporativa estándar con rotación automática de credenciales y auditoría avanzada.	Alta complejidad de infraestructura (requiere mantener un clúster de Vault).

Justificación: Se seleccionó **Bitnami Sealed Secrets** debido a que permite un flujo de trabajo puramente declarativo: los desarrolladores pueden cifrar credenciales usando la CLI `kubeseal` y commitear el archivo al repositorio Git. El operador que reside en el clúster es el único capaz de descifrarlo en memoria y generar el objeto `Secret` local.

Prueba de Reconstrucción desde Cero: Se eliminó el clúster con `k3d cluster delete com600` y se siguió al pie de la letra el documento `README.md`. El clúster y el microservicio completo se reconstruyeron y quedaron operativos en **menos de 3 minutos** sin requerir intervención manual adicional.

4. CONCLUSIONES

1. El paradigma declarativo es la base indispensable de la infraestructura moderna, permitiendo idempotencia, control de versiones y auditoría formal de cada recurso desplegado.
2. Las sondas de salud (readiness y liveness) no son opcionales: son el mecanismo indispensable para permitir actualizaciones progresivas sin corte de servicio y prevenir reinicios destructivos en cadena ante fallas transitorias de dependencias.
3. Helm simplifica y unifica la administración de despliegues multi-entorno, evitando la dispersión de manifiestos y permitiendo rollbacks atómicos de aplicaciones complejas.
4. GitOps con Argo CD cierra el ciclo devops de forma definitiva, eliminando la deriva de

configuración y convirtiendo a Git en la fuente indiscutible de verdad para la infraestructura de producción.